CSE3015 – AWS Cloud Practitioner

MODULE 3 — PART 1

AWS Networking & Content Delivery

CONCEPT-FOCUSED STUDY NOTES

Route 53 • API Gateway • VPC • Subnets • Route Tables • Security Groups • NAT Gateway

Table of Contents

1. Amazon Route 53
   DNS Fundamentals · Hosted Zones · DNS Records · Health Checks · Routing Policies
2. Amazon API Gateway
   API Types · Integrations · Throttling & Caching · Comparisons
3. Amazon VPC
   Core Components · Architecture · Multi-Tier Application Design
4. VPC Subnets
   CIDR Blocks · Public vs Private Subnets · Availability Zones
5. Route Tables
   Routes · Targets · 0.0.0.0/0 · Public vs Private Routing
6. Security Groups
   Stateful Firewalls · Inbound/Outbound Rules · SG vs NACL
7. NAT Gateway
   Outbound-Only Internet Access · NAT Gateway vs NAT Instance
8. Module 3 – How It All Fits Together (Final Architecture)

1. Amazon Route 53

DNS Fundamentals (Prerequisite Concepts)

> **DNS (Domain Name System)** is the distributed naming system that translates human-readable domain names (e.g. `www.example.com`) into machine-readable IP addresses (e.g. `192.0.2.10`).

- **Why DNS is required**
  - Computers communicate using IP addresses, not names.
  - Humans cannot easily remember numeric IP addresses.
  - DNS acts as the "phonebook of the internet".
- **Domain name** — a human-friendly address for a resource (e.g. `amazon.com`).
- **IP address** — the actual numeric network address a computer uses to route traffic.
- **DNS resolution** — the overall process of converting a domain name into an IP address.
- **DNS resolver** — a server (often run by an ISP or public service) that receives a client's query and finds the answer on its behalf.
- **DNS hierarchy** — DNS is organized as an inverted tree: Root → Top-Level Domain (TLD, e.g. `.com`) → Authoritative domain (e.g. `example.com`) → Subdomain (e.g. `www`).
- **Authoritative DNS server** — the server that holds the actual, final DNS records for a domain and gives the definitive answer.
- **Name servers (NS)** — servers that are delegated authority to answer for a specific domain.
- **Recursive resolution** — a resolver queries multiple servers step-by-step (root → TLD → authoritative) on behalf of the client until it gets the final answer, then caches and returns it.

Diagram: DNS Resolution Flow



Amazon Route 53

> **Definition:** Amazon Route 53 is AWS's highly available and scalable managed **DNS web service**, also used for **domain registration** and **traffic routing/health checking**.

- **In simple language:** Route 53 is AWS's phonebook service — it tells the internet which IP address (or AWS resource) a domain name points to, while also letting you buy domains and intelligently route traffic.
- **Why Route 53 is used**
  - To manage DNS records for domains without running your own DNS servers.
  - To route users to the closest, healthiest, or best-performing endpoint automatically.
  - To integrate DNS tightly with other AWS resources (ELB, S3, CloudFront) via Alias records.
  - Name "53" refers to port 53, the standard port used for DNS traffic.
- **Route 53 architecture (high level)**
  - Domain registration component — register/manage domain names.
  - Hosted zone component — stores DNS records for a domain.
  - Routing policy engine — decides which record to return based on rules.
  - Health checking component — monitors endpoint health and feeds routing decisions.

Hosted Zones

Type	Description	When Used
Public Hosted Zone	Controls how traffic is routed on the public internet for a domain.	Public-facing websites/applications reachable from anywhere.
Private Hosted Zone	Controls DNS resolution only within one or more associated VPCs.	Internal applications, microservices, or resources that should not be resolvable from the public internet.

- A hosted zone is essentially a **container for DNS records** belonging to a domain.
- Private hosted zones require explicit VPC association and DNS resolution/hostnames enabled on the VPC.

DNS Records in Route 53

Record	Stores / Meaning	Typical Use	Example
A	Maps a domain name to an IPv4 address	Point a domain directly to a server's IP	`example.com → 192.0.2.10`
AAAA	Maps a domain name to an IPv6 address	Same as A record but for IPv6	`example.com → 2001:db8::1`
CNAME	Maps a domain name to another domain name (not an IP)	Alias a subdomain to another hostname (cannot be used at zone apex)	`shop.example.com → shop.myapp.io`
Alias Record	Route 53–specific record that maps a name directly to an AWS resource	Point root/zone-apex domain to ELB, CloudFront, S3, API Gateway — free, and resolves at the DNS level	`example.com → ALB DNS name`
NS	Lists the authoritative name servers for a domain/hosted zone	Delegation of a domain or subdomain	`ns-123.awsdns-45.com`
SOA	Start of Authority — administrative information about the zone (primary NS, admin email, serial number, refresh/retry timers)	Automatically created with every hosted zone	—
TTL	Time To Live — how long (in seconds) a resolver may cache a record before re-querying	Controls propagation speed of DNS changes	`TTL = 300` (5 minutes)

> **Commonly Confused: CNAME vs Alias Record**
> - **CNAME** cannot be used at the zone apex (e.g. `example.com`) — only on subdomains.
> - **Alias** can be used at the zone apex, is free of charge, and Route 53 resolves it internally to the target AWS resource without an extra DNS lookup.
> - Alias records also automatically track changes to the underlying resource's IP (e.g. when an ELB's IP changes).

Route 53 Health Checks

- **Purpose:** continuously monitor the health/availability of an endpoint (IP address, domain, or CloudWatch alarm).
- **How health checks work**
  1. Route 53 sends automated requests to the configured endpoint at set intervals.
  2. The endpoint must respond within a threshold (e.g. HTTP 200) to be considered "healthy".
  3. If a set number of checks fail, the endpoint is marked "unhealthy".
  4. Route 53 stops routing traffic to unhealthy endpoints (depending on routing policy).
- **Relationship with routing:** Health check status is fed into routing policies (especially Failover, Weighted, and Multivalue) so Route 53 avoids sending users to unhealthy resources.
- **Failover concept:** A primary resource is used under normal conditions; if its health check fails, Route 53 automatically starts responding with the secondary (backup) resource's record instead.

Routing Policies

1. Simple Routing

- **Definition:** Maps one domain name to one resource (or a fixed set with no health-based logic); no special routing logic applied.
- **How it works:** Returns the record value(s) as-is; if multiple values are given, one is chosen randomly by the client.
- **Example:** `example.com → 192.0.2.10`
- **Use case:** Single-server websites, basic setups with no need for failover.

2. Weighted Routing

- **Definition:** Distributes traffic across multiple resources according to assigned weights.
- **How it works:** Each record gets a relative weight; probability of being chosen = weight ÷ sum of all weights.
- **Example:** Server A weight 70, Server B weight 30 → ~70% of traffic to A, ~30% to B.
- **Use case:** A/B testing, canary/gradual deployments, blue-green rollouts.

3. Latency-Based Routing

- **Definition:** Routes users to the AWS region that provides the **lowest network latency**.
- **How it works:** Uses latency measurements between users and AWS regions, not physical distance.

- **Example:** A user in Singapore is routed to the ap-southeast-1 endpoint instead of us-east-1 if it has lower latency.
- **Use case:** Globally distributed applications where response speed matters.

4. Failover Routing

- **Definition:** Routes traffic to a primary resource, and automatically switches to a secondary resource if the primary is unhealthy.
- **How it works:** Requires a health check on the primary record; failover is automatic and based on health check status.
- **Example:** Primary web server in Mumbai; failover to a DR (disaster recovery) site in Singapore if Mumbai goes down.
- **Use case:** Disaster recovery and high-availability architectures.

5. Geolocation Routing

- **Definition:** Routes traffic based on the **geographic location of the user** (continent, country, or state).
- **How it works:** Route 53 checks the source of the DNS query and matches it to the most specific configured location record.
- **Example:** Users in the EU are routed to an EU endpoint to comply with data-residency requirements.
- **Use case:** Content localization, regulatory/compliance restrictions, restricting content by country.

6. Geoproximity Routing

- **Definition:** Routes traffic based on the geographic location of users **and** resources, with the ability to shift more or less traffic using a "bias" value. (Requires Route 53 Traffic Flow.)
- **How it works:** Draws a bias-adjusted geographic boundary around each resource; increasing bias expands the resource's traffic "pull" radius.
- **Example:** Shifting more traffic toward a Mumbai data center by increasing its bias value, even for users slightly closer to Singapore.
- **Use case:** Fine-grained control over traffic distribution across regional resources.

7. Multivalue Answer Routing

- **Definition:** Returns multiple healthy IP addresses (up to 8) in response to a DNS query, chosen at random.
- **How it works:** Combines multiple records with optional health checks; only healthy records are returned.
- **Example:** Three web servers configured; Route 53 returns all three healthy IPs, client picks one.
- **Use case:** Simple client-side load balancing with basic health awareness (not a replacement for a full load balancer).

Comparison: Route 53 Routing Policies

Policy	Decision Basis	Health Check Aware?	Best For
Simple	Fixed/random	No	Single resource, no logic needed
Weighted	Assigned weight ratio	Optional	A/B testing, gradual rollout
Latency-based	Lowest network latency	Optional	Global apps needing speed
Failover	Primary/secondary health	Yes (required)	Disaster recovery
Geolocation	User's geographic location	Optional	Localization/compliance
Geoproximity	Location + bias	Optional	Fine-tuned regional traffic shift
Multivalue Answer	Multiple healthy records	Yes (recommended)	Simple client-side load balancing

Real-World Examples

- **E-commerce site with global customers:** Uses latency-based routing so users worldwide connect to the nearest low-latency region.
- **Banking application with DR requirements:** Uses failover routing with health checks to switch to a backup data center during an outage.
- **SaaS company running a canary release:** Uses weighted routing to send 5% of traffic to a new backend version before a full rollout.

Use Cases

- Domain registration and management for a business website.
- DNS failover for high-availability applications.
- Global traffic distribution for multi-region deployments.
- Private internal DNS for microservices inside a VPC.
- Health monitoring of public endpoints.

Advantages

- Highly available and scalable (100% availability SLA).
- Tight integration with AWS resources via Alias records (no extra cost).
- Multiple intelligent routing policies for different business needs.
- Built-in health checking and automatic failover.
- Pay-as-you-go pricing; no infrastructure to manage.

Limitations / Disadvantages

- DNS propagation depends on TTL — changes are not instantaneous everywhere.
- Geolocation accuracy depends on the resolver's IP database, which is not always precise.
- Complex routing setups (geoproximity, traffic flow policies) have a steeper learning curve.
- Cannot control application-layer load balancing decisions (that is the role of ELB).

Important Terminology

Term	Meaning
Hosted Zone	Container for DNS records of a domain
TTL	Cache duration for a DNS record
Alias Record	Route 53 record mapping a name to an AWS resource
Health Check	Automated monitor of an endpoint's availability
Routing Policy	Rule set determining which record Route 53 returns
Zone Apex	The root domain itself (e.g. `example.com`, no subdomain)

Related AWS Services

- **Elastic Load Balancing (ELB)** — Route 53 Alias records often point to an ELB.
- **CloudFront** — Route 53 can route a custom domain to a CloudFront distribution.
- **S3** — Route 53 can route a domain to a static website hosted on S3.
- **API Gateway** — Custom domains for APIs are mapped via Route 53.
- **CloudWatch** — Health checks can be based on CloudWatch alarms.

Important Relationships (Module 3 Context)

```
User types domain name
        |
        v
   Route 53 (DNS)
        |
        v
Resolves to: API Gateway / Load Balancer endpoint
        |
        v
Traffic enters the target AWS architecture (e.g. VPC)
```

Route 53 is the **entry point** for Module 3's architecture — it is the first service a user's request touches before reaching API Gateway, a Load Balancer, or any resource inside a VPC.

Commonly Confused: Public Hosted Zone vs Private Hosted Zone

- Public hosted zone answers DNS queries from anywhere on the internet.
- Private hosted zone answers DNS queries only from within associated VPC(s) — used for internal-only resolution.

2. Amazon API Gateway

Definition: Amazon API Gateway is a fully managed AWS service that lets developers create, publish, secure, monitor, and maintain APIs at any scale, acting as the "front door" for applications to access backend services.

What Is an API?

- **API (Application Programming Interface)** — a defined contract that lets one piece of software request services/data from another.
- **Why APIs are required**
  - They decouple frontend clients from backend implementation details.
  - They provide a standard, predictable way for different systems (web, mobile, IoT) to communicate.
  - They allow backend logic to change without breaking client integrations, as long as the contract is preserved.

Why API Gateway Is Needed

- Exposing backend compute (Lambda, EC2, containers) directly to the internet is insecure and hard to manage.
- Without API Gateway, developers would need to build authentication, throttling, caching, and monitoring manually for every backend.
- API Gateway centralizes these cross-cutting concerns in a single managed layer.

API Gateway Architecture — Core Concepts

Component	Purpose
Client	The consumer of the API — browser, mobile app, another service
API Endpoint	The public URL clients call to reach the API
Resource	A logical object/path in the API (e.g. `/users`, `/orders`)
Method	An HTTP verb attached to a resource (GET, POST, PUT, DELETE, etc.)
Integration	The backend that a method is connected to (Lambda, HTTP endpoint, AWS service)
Request / Response	Data sent by the client and returned by the backend, which API Gateway can transform
Stage	A named, versioned snapshot of the API configuration (e.g. `dev`, `prod`)
Deployment	The action of publishing the current API configuration to a stage
Backend	The actual compute/service that fulfills the request (Lambda, EC2, DynamoDB, etc.)

- Resources and methods together define the API's surface (e.g. `GET /users/{id}`).
- Stages allow the same API definition to exist in multiple environments simultaneously.
- A deployment must be made for changes to a stage to become live.

API Types

Type	Description	Typical Use
REST API	Full-featured API type supporting request validation, API keys, usage plans, per-method transformations, and detailed configuration	Enterprise-grade APIs needing fine-grained control
HTTP API	A lighter, lower-latency, lower-cost alternative to REST APIs with a reduced feature set	Simple proxy APIs to Lambda or HTTP backends where cost/speed matter more than advanced features
WebSocket API	Maintains persistent, two-way connections between client and server	Real-time applications: chat apps, live dashboards, multiplayer games

Comparison: REST API vs HTTP API vs WebSocket API

Feature	REST API	HTTP API	WebSocket API
Connection type	Request/response	Request/response	Persistent, bi-directional
Cost	Higher	Lower (~70% cheaper)	Based on connection minutes + messages
Latency	Standard	Lower	Real-time
Feature richness	Very high (API keys, usage plans, request validation)	Basic (JWT auth, simple routing)	Route-based message handling
Use case	Complex, enterprise APIs	Simple proxy/serverless APIs	Chat, live feeds, gaming

Integrations

API Gateway → AWS Lambda (Serverless)

```
Client
  |
  v
API Gateway  --(invokes)-->  AWS Lambda  --(reads/writes)-->  DynamoDB
  |                              |
  +----------- Response <--------+
```

API Gateway → EC2 / Backend Servers

```
Client
  |
  v
API Gateway --(HTTP proxy)--> Application Load Balancer --> EC2 instances (in VPC)
```

API Gateway → Other AWS Services (Direct Integration)

```
Client --> API Gateway --(direct service integration)--> S3 / SQS / Step Functions
```

- API Gateway can integrate directly with many AWS services without needing a Lambda function in between, reducing cost and complexity for simple pass-through operations.
- For private backends inside a VPC (e.g. internal ALB, EC2), a **VPC Link** is used to connect API Gateway securely without exposing the backend publicly.

Other Core Concepts

- **Authentication** — verifying who the caller is (e.g. via IAM, Cognito User Pools, or Lambda authorizers).
- **Authorization** — verifying what the authenticated caller is allowed to do (e.g. IAM policies, custom authorizers, scopes).
- **Throttling** — limiting the rate of requests (requests per second) to protect backend systems from being overwhelmed; configurable per API/stage/method via usage plans.
- **Caching** — API Gateway can cache responses for a configurable TTL, reducing backend calls and latency for repeated requests.
- **Monitoring** — integrates with CloudWatch for metrics (latency, error rates, request counts) and logging.
- **Error handling** — API Gateway can map backend errors to custom HTTP status codes/response bodies via gateway responses and integration responses.

How It Works — Step by Step

Example Architecture: Client → API Gateway → Lambda → Database

```
Client
  |
  v
API Gateway
  |
  v
Lambda
  |
  v
Database
```

1. Client sends an HTTPS request to the API Gateway endpoint (e.g. `POST /orders`).
2. API Gateway checks authentication/authorization (e.g. validates a JWT or IAM signature).
3. API Gateway applies throttling limits and checks the response cache (if enabled).
4. Request is transformed (if configured) and forwarded to the integrated backend — here, an AWS Lambda function.
5. Lambda executes business logic and reads/writes to the database (e.g. DynamoDB or RDS inside a VPC).
6. Lambda returns a response to API Gateway.
7. API Gateway transforms the response (if configured) and sends it back to the client.
8. CloudWatch logs and metrics are recorded for the request.

Real-World Examples

- **Mobile banking app:** Uses a REST API on API Gateway with API keys and usage plans to expose account and transaction endpoints backed by Lambda functions.
- **Serverless e-commerce backend:** Uses an HTTP API for low-cost, low-latency product catalog and checkout endpoints backed by Lambda and DynamoDB.
- **Live customer support chat:** Uses a WebSocket API to maintain persistent connections between agents and customers for real-time messaging.

Use Cases

- Building serverless backends for web and mobile apps.
- Exposing microservices through a single unified API surface.
- Rate-limiting and securing access to internal services.
- Real-time applications using WebSocket APIs.
- API monetization using API keys and usage plans.

Advantages

- Fully managed — no servers to provision or patch.
- Built-in security, throttling, and monitoring.
- Scales automatically with traffic.
- Tight integration with Lambda for serverless architectures.
- Supports multiple API styles (REST, HTTP, WebSocket).

Limitations / Disadvantages

- Can introduce additional latency compared to direct backend access.
- Cost can grow significantly at very high request volumes.
- REST API feature richness comes with added configuration complexity.
- Payload size and timeout limits apply (e.g. 29-second integration timeout, 10 MB payload limit).

Important Terminology

Term	Meaning
Stage	Named deployment environment of an API (dev/test/prod)
Throttling	Limiting request rate to protect backend
VPC Link	Private connection from API Gateway to resources inside a VPC
Usage Plan	Defines API key access, quotas, and throttling limits
Mapping Template	Transforms request/response payloads between client and backend format

Related AWS Services

- **AWS Lambda** — most common serverless backend integration.
- **Amazon Cognito** — user authentication for APIs.
- **CloudWatch** — monitoring, logging, alarms.
- **VPC (via VPC Link)** — connecting to private backend resources.
- **Route 53** — custom domain names for APIs.

Comparisons

API Gateway vs Application Load Balancer (ALB)

Aspect	API Gateway	Application Load Balancer
Layer	Application layer, API-focused	Application layer, traffic distribution-focused
Features	Auth, throttling, caching, transformations, API keys	Path/host-based routing, health checks, SSL termination
Serverless-friendly	Yes — native Lambda integration	Requires Lambda target group support
Cost model	Pay per request	Pay per load balancer hour + LCU
Best for	API-centric, serverless workloads	General HTTP(S) traffic distribution to servers/containers

API Gateway vs Direct Backend Access

Aspect	Via API Gateway	Direct Backend Access
Security	Centralized auth, throttling	Must be implemented manually per backend
Backend exposure	Backend can remain private	Backend often must be public
Monitoring	Centralized via CloudWatch	Must be built separately
Latency	Slight added hop	Lower (direct)

Important Relationships (Module 3 Context)

```
Route 53 (DNS)
     |
     v
API Gateway (or ALB)
     |
     v
VPC  --> Private Subnet --> Lambda / EC2 --> Database
```

API Gateway commonly sits just after Route 53 in the traffic path, acting as the controlled entry point into an application before requests reach compute resources inside a VPC.

3. Amazon VPC

This is a core, high-weightage topic — all other Module 3 topics (Subnets, Route Tables, Security Groups, NAT Gateway) are components of a VPC.

> **Definition:** Amazon Virtual Private Cloud (VPC) is a logically isolated virtual network within AWS where you launch AWS resources, with full control over IP addressing, subnetting, routing, and security.

Purpose & Fundamentals

- **Virtual networking:** A VPC is software-defined networking — it mimics a traditional physical data-center network (subnets, routers, firewalls) but is entirely virtualized inside AWS.
- **AWS networking fundamentals**
  - Every VPC lives inside one AWS Region and can span multiple Availability Zones (AZs).
  - A VPC is isolated by default — nothing outside it can reach resources inside unless explicitly allowed.
- **Default VPC:** AWS automatically creates one per region with public subnets in every AZ, pre-configured for immediate internet access — convenient for quick starts, not recommended for production.
- **Custom VPC:** A VPC you design yourself — custom CIDR block, subnets, routing, and security — used for production-grade, intentional architectures.

Addressing Concepts

- **CIDR (Classless Inter-Domain Routing):** A notation (e.g. `10.0.0.0/16`) that defines a block of IP addresses; the number after "/" is the prefix length, indicating how many bits are fixed for the network portion.
- **IPv4:** 32-bit addressing scheme; the traditional and most commonly used addressing method in VPCs.
- **IPv6:** 128-bit addressing scheme; optional in a VPC, provides a much larger address space, all IPv6 addresses in AWS VPCs are globally unique (public) by design.
- **Private IP:** An address only reachable within the VPC (or connected private networks) — not routable on the public internet.
- **Public IP:** An address reachable from the internet; can be auto-assigned and changes if the instance is stopped/started.
- **Elastic IP:** A static, persistent public IPv4 address that you allocate to your account and can attach/detach from instances as needed — unlike a normal public IP, it doesn't change.

Why VPC Is Needed

- Without a VPC, AWS resources would have no network isolation or boundary — anyone could potentially reach them.
- Enterprises need to replicate on-premises network segmentation practices (public-facing tier vs private internal tier) in the cloud.
- Provides control over IP ranges, routing, and traffic filtering, which is essential for security and compliance.

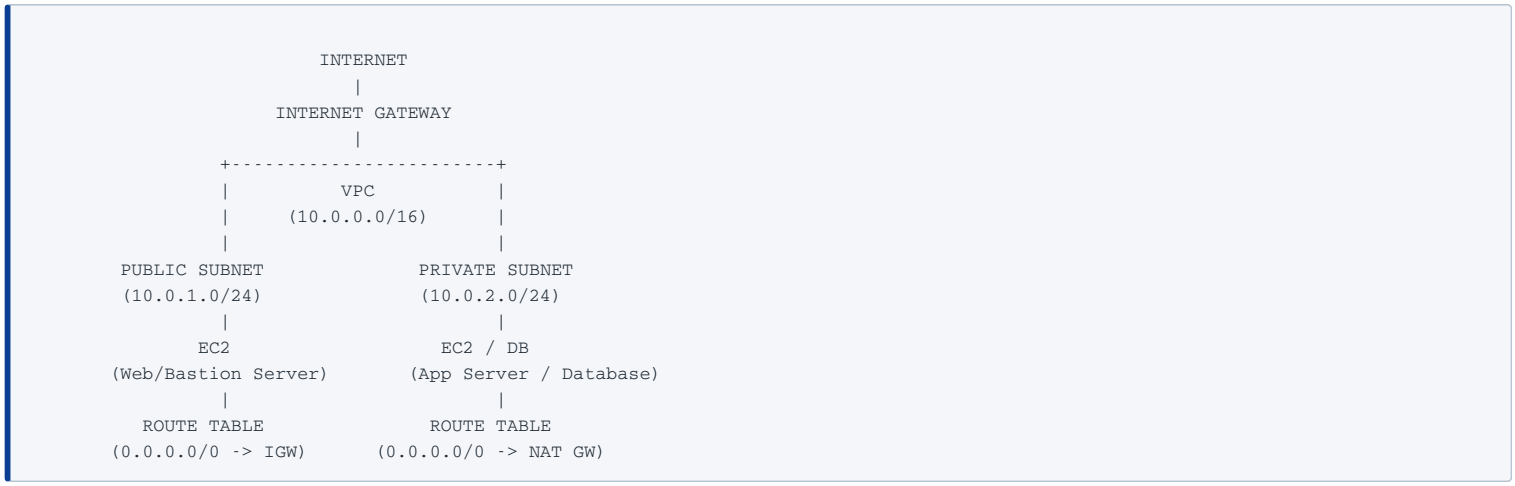
Key VPC Components

Component	Purpose
Subnet	A segment of the VPC's IP range tied to one Availability Zone
Route Table	Set of rules that determine where network traffic is directed
Internet Gateway (IGW)	Allows communication between the VPC and the public internet
NAT Gateway	Allows private subnet resources to initiate outbound internet traffic without being publicly reachable
Security Group	Stateful, instance-level virtual firewall
Network ACL (NACL)	Stateless, subnet-level virtual firewall

Each of these is explored in depth as a dedicated topic later in this module — here they are introduced as parts of the overall VPC picture.

VPC Architecture

Basic VPC Layout: Public + Private Subnet



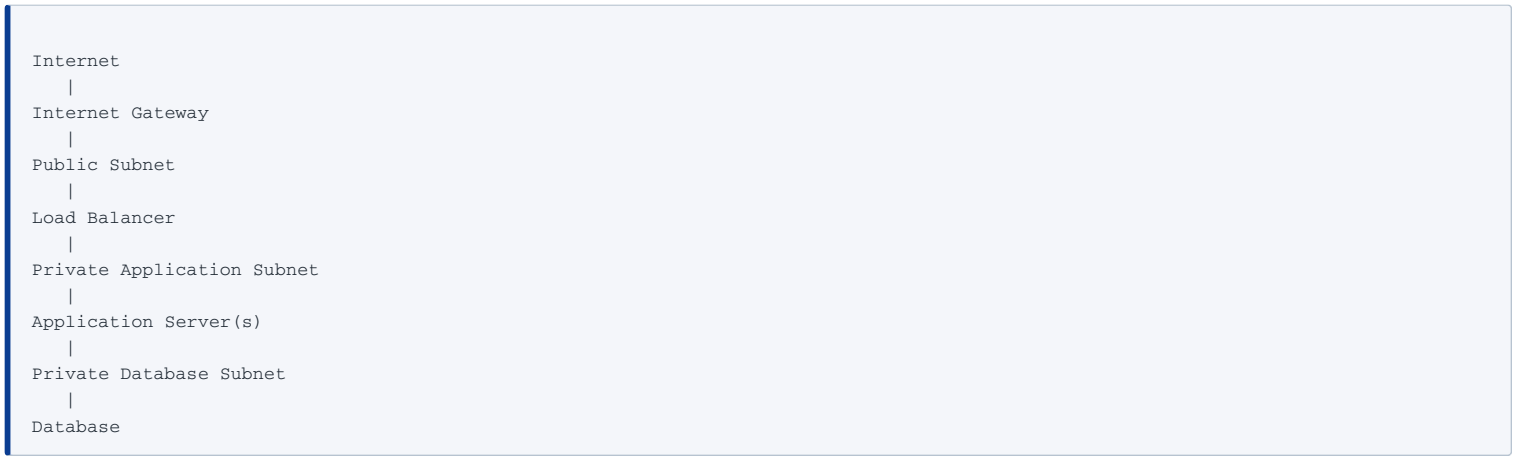
- The VPC itself is defined by a single top-level CIDR block (e.g. `10.0.0.0/16`).
- Subnets are carved out of that CIDR block and each lives entirely within one AZ.
- The Internet Gateway is attached to the VPC (not to any specific subnet) and enables internet connectivity.
- Whether a subnet is "public" or "private" is determined entirely by its route table (explained in the Subnets topic).

How It Works — Traffic Flow Example

1. A user's request reaches the Internet Gateway attached to the VPC.
2. The Internet Gateway forwards the traffic based on the destination IP.
3. The public subnet's route table sends internet-bound and inbound traffic through the IGW to the target EC2 instance.
4. Security Groups on the EC2 instance check whether the inbound traffic is permitted.
5. If the request needs to reach a private backend, it is forwarded internally within the VPC (private IP routing) to the private subnet.
6. Return traffic follows the reverse path back to the user.

Multi-Tier Application Architecture

3-Tier Application inside a VPC



- **Why each component exists**
  - Public subnet + Load Balancer: single controlled internet-facing entry point.
  - Private application subnet: business logic isolated from direct internet exposure.
  - Private database subnet: most sensitive tier, given the strictest isolation.
- **Why different tiers are separated**
  - Limits the "blast radius" if one tier is compromised.
  - Enables tier-specific security rules (e.g. database only accepts traffic from the app tier, never from the internet).
  - Matches the principle of least privilege at the network level.
- **How traffic flows:** Internet → IGW → Load Balancer (public subnet) → App servers (private subnet) → Database (private subnet, most restricted).
- **How security is improved:** Only the load balancer is internet-reachable; app and database tiers have no direct route to/from the internet, drastically reducing the attack surface.

Real-World Examples

- **Startup web application:** A single VPC with one public subnet (web servers behind a load balancer) and one private subnet (database), isolating the database from direct internet access.
- **Enterprise multi-tier system:** A VPC spanning multiple AZs with separate public, app, and database subnet tiers replicated per AZ for high availability.

- **Hybrid cloud company:** Uses a VPC connected to an on-premises data center via VPN/Direct Connect, extending the corporate network into AWS.

Use Cases

- Hosting secure, isolated multi-tier web applications.
- Creating separate environments (dev/test/prod) using separate VPCs.
- Isolating sensitive workloads (databases, internal tools) from the public internet.
- Hybrid connectivity between on-premises networks and AWS.

Advantages

- Full control over IP addressing, subnetting, and routing.
- Strong network isolation and security boundary by default.
- Highly customizable (route tables, gateways, peering, endpoints).
- Supports hybrid connectivity (VPN, Direct Connect) and VPC-to-VPC peering.

Limitations / Disadvantages

- Requires careful planning of CIDR ranges to avoid future IP conflicts (especially with peering/hybrid setups).
- Complex architectures (multiple subnets, route tables, gateways) require networking expertise to design correctly.
- Some AWS services and resource limits (e.g. subnets per VPC, route table entries) impose design constraints.

Important Terminology

Term	Meaning
CIDR Block	Range of IP addresses assigned to the VPC/subnet
Availability Zone (AZ)	An isolated physical data center location within a region
Internet Gateway	VPC component enabling internet connectivity
Elastic IP	Static, persistent public IPv4 address
Default VPC	Auto-created VPC ready for immediate use

Related AWS Services

- **EC2** — compute instances launched inside VPC subnets.
- **RDS** / **DynamoDB** — databases often placed in private subnets.
- **Elastic Load Balancing** — distributes traffic across instances in a VPC.
- **AWS Direct Connect** / **VPN** — hybrid connectivity to a VPC.
- **VPC Peering** / **Transit Gateway** — connecting multiple VPCs together.

Important Relationships (Module 3 Context)

```
VPC
 -> Subnet
 -> Route Table
 -> Internet Gateway
 -> NAT Gateway
 -> Security Group
```

- A **VPC** is the outer container; everything else in Module 3 exists inside or attached to it.
- **Subnets** divide the VPC's CIDR block into smaller network segments across AZs.
- **Route Tables** attached to subnets decide where traffic goes (internet, NAT, local, etc.).
- The **Internet Gateway** is what a public subnet's route table points to for direct internet access.
- The **NAT Gateway** is what a private subnet's route table points to for outbound-only internet access.
- **Security Groups** attach directly to resources (like EC2 instances) inside the subnets to filter traffic at the instance level.

Commonly Confused: Default VPC vs Custom VPC

- Default VPC comes pre-configured with public subnets and internet access in every AZ — good for quick testing.
- Custom VPC is built intentionally by the user/architect with tailored CIDR, subnet, and routing design — recommended for production workloads.

4. VPC Subnets

Definition: A subnet is a range of IP addresses within a VPC's CIDR block, tied to exactly one Availability Zone, used to logically segment and organize resources.

Purpose

- Break a large VPC network into smaller, manageable, and independently securable segments.
- Allow different resources (web servers, app servers, databases) to be placed in appropriately isolated network zones.
- Enable resource placement across multiple Availability Zones for fault tolerance.

Core Concepts

- **CIDR blocks:** Each subnet has its own CIDR range, which must be a subset of (and non-overlapping within) the parent VPC's CIDR block.
- **IP address allocation:** AWS reserves 5 IP addresses in every subnet (network address, VPC router, DNS, future use, and broadcast address placeholder) — these are not usable by resources.
- **Subnet size:** Determined by the CIDR prefix length; smaller prefix number (e.g. /24) = larger address range than a larger prefix (e.g. /28).
- **Availability Zones (AZs):** Isolated data centers within a region; a subnet must be created within exactly one AZ and cannot span multiple AZs.
- **Subnet-to-AZ relationship:** One subnet → one AZ, but one AZ can contain multiple subnets.
- **Route table association:** Every subnet must be associated with exactly one route table (either the main route table or a custom one), which determines its traffic behavior.

Important concept: "A subnet is not public simply because it is *named* public subnet."

- AWS does not have a special "public subnet" resource type — "public" and "private" are just labels people use.
- What actually makes a subnet public is its **route table**: if it has a route sending `0.0.0.0/0` traffic to an **Internet Gateway**, it behaves as a public subnet.
- A subnet named "public-subnet" with no IGW route in its route table is, functionally, a private subnet — naming has no effect on actual behavior.
- Therefore: routing configuration determines internet connectivity, not naming conventions.

CIDR Notation Explained

- **CIDR notation:** Written as `IP_address/prefix_length` (e.g. `10.0.0.0/16`).
- **Prefix length:** Number of bits fixed as the "network" portion; remaining bits are available for host addresses.
- **Network address:** The base address of the range (all host bits set to 0).
- **Host addresses:** The usable IP addresses within that range for resources.
- **Subnet ranges:** Smaller subnets are carved out from the larger VPC range by increasing the prefix length.

CIDR Examples

CIDR	Meaning	Approx. Usable Addresses
`10.0.0.0/16`	VPC-level range (large block)	65,536 total addresses
`10.0.1.0/24`	Subnet 1, carved from the VPC range	256 addresses (251 usable after AWS reservations)
`10.0.2.0/24`	Subnet 2, carved from the VPC range	256 addresses (251 usable)

Relationship: VPC CIDR Split into Subnets

```
VPC CIDR: 10.0.0.0/16   (65,536 addresses)
   |
   +-- Subnet A: 10.0.1.0/24   (Public Subnet - AZ-a)
   |
   +-- Subnet B: 10.0.2.0/24   (Private Subnet - AZ-a)
   |
   +-- Subnet C: 10.0.3.0/24   (Public Subnet - AZ-b)
   |
   +-- Subnet D: 10.0.4.0/24   (Private Subnet - AZ-b)
```

Each subnet's range is a non-overlapping slice of the parent VPC's CIDR block; together they must stay within the VPC's total address space.

Public vs Private Subnet

Architecture: Public and Private Subnets



Aspect	Public Subnet	Private Subnet
Route to internet	Direct, via Internet Gateway	Indirect (outbound-only), via NAT Gateway, or none
Publicly reachable?	Yes, if resource has a public IP	No
Typical resources	Load balancers, bastion hosts, NAT Gateway itself	Application servers, databases
Security exposure	Higher — internet-facing	Lower — shielded from direct internet access

- **Traffic flow (public):** Internet → IGW → Public Subnet Route Table → Resource (if SG/NACL allow).
- **Traffic flow (private, outbound):** Private Resource → Private Subnet Route Table → NAT Gateway (in public subnet) → IGW → Internet.
- **Traffic flow (private, inbound from internet):** Not possible by default — there is no route from the internet directly into a private subnet.

Real-World Examples

- **Web application:** Public subnet hosts the load balancer; private subnet hosts application servers that only accept traffic from the load balancer.
- **Database tier:** A private subnet with no route to the internet at all, hosting an RDS instance only reachable from the app tier.
- **Bastion host setup:** A small EC2 instance in a public subnet used as a secure jump box to access resources in private subnets via SSH.

Use Cases

- Separating internet-facing components from internal/backend components.
- Distributing resources across AZs for high availability within a subnet tier.
- Enforcing compliance requirements that mandate isolated network segments for sensitive data.

Advantages

- Granular control over which resources can be reached from the internet.
- Enables clear network segmentation aligned with application tiers.
- Supports high availability by spreading subnets across multiple AZs.

Limitations / Disadvantages

- Subnet CIDR sizes are fixed at creation time and cannot be resized later (must create a new subnet).
- Poor subnet/CIDR planning can lead to IP exhaustion or wasted address space.
- Managing many subnets across AZs and tiers adds operational complexity.

Important Terminology

Term	Meaning
CIDR Block	Range of IP addresses assigned to a subnet
Availability Zone	Isolated data center location a subnet belongs to
Public Subnet	Subnet whose route table sends 0.0.0.0/0 to an Internet Gateway
Private Subnet	Subnet with no direct route to an Internet Gateway

Related AWS Services

- **Route Tables** — determine a subnet's actual public/private behavior.
- **Internet Gateway** — target for public subnet routing.
- **NAT Gateway** — typically deployed inside a public subnet to serve private subnets.
- **Security Groups / NACLs** — apply additional traffic filtering within subnets.

Important Relationships (Module 3 Context)

Subnets are the direct child of a VPC and the direct parent of a Route Table association: **VPC → Subnet → Route Table → (Internet Gateway or NAT Gateway)**. The subnet itself only defines an IP range and an AZ — the route table is what gives it public or private behavior.

Commonly Confused: Public Subnet vs Private Subnet

- Determined entirely by the associated route table's routes, not by name or by whether resources inside have public IPs.
- A resource can have a public IP but still be unreachable if the subnet's route table lacks an IGW route.

5. Route Tables

> **Definition:** A route table is a set of rules (routes) associated with a subnet or VPC that determines where network traffic from that subnet is directed.

Purpose

- Controls how traffic leaving a subnet is forwarded — to the internet, to other subnets, to a NAT Gateway, to peered VPCs, or nowhere at all.
- Is the single deciding factor for whether a subnet behaves as "public" or "private".

Core Concepts

- **Route:** A single rule mapping a destination CIDR range to a target.
- **Destination:** The IP range the rule applies to (e.g. `10.0.0.0/16` for local traffic, `0.0.0.0/0` for "everything else").
- **Target:** Where matching traffic should be sent — e.g. `local`, an Internet Gateway, a NAT Gateway, a peering connection.
- **Main route table:** The default route table automatically assigned to a VPC; any subnet not explicitly associated with another table uses this one.
- **Custom route table:** A route table you create and explicitly associate with specific subnets for tailored routing behavior.
- **Subnet association:** Each subnet must be linked to exactly one route table at a time.
- **Local route:** An automatic, non-removable route that allows all resources within the VPC's CIDR range to communicate with each other.
- **Internet Gateway route:** A route sending internet-bound traffic (`0.0.0.0/0`) to the VPC's Internet Gateway.
- **NAT Gateway route:** A route sending internet-bound traffic (`0.0.0.0/0`) to a NAT Gateway instead of directly to the IGW.
- **Default route:** Commonly refers to the `0.0.0.0/0` entry — the "catch-all" rule for any destination not otherwise matched.

0.0.0.0/0 → Internet Gateway vs 0.0.0.0/0 → NAT Gateway

Route	Meaning	Effect
`0.0.0.0/0 → Internet Gateway`	All non-local traffic goes directly to/from the internet	Subnet is **public**; resources with public IPs are directly reachable both ways
`0.0.0.0/0 → NAT Gateway`	All non-local outbound traffic is routed through a NAT Gateway	Subnet is **private**; resources can initiate outbound connections but cannot be reached from the internet

- The destination (`0.0.0.0/0`) is identical in both cases — the difference is entirely in the **target**.
- Routing through an IGW allows bidirectional internet communication.
- Routing through a NAT Gateway only allows outbound-initiated communication; unsolicited inbound connections from the internet are still blocked.

Route Table Examples

Public Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Internet Gateway (igw-xxxx)

Private Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	NAT Gateway (nat-xxxx)

Traffic Flow Diagrams

Public Subnet Traffic Flow

```
Internet
   |
Internet Gateway
   |
Public Subnet Route Table (0.0.0.0/0 -> IGW)
   |
EC2 instance (public IP)
```

Private Subnet Traffic Flow (Outbound Only)

```
Private EC2 instance
   |
Private Subnet Route Table (0.0.0.0/0 -> NAT GW)
   |
NAT Gateway (sits in Public Subnet)
   |
Internet Gateway
   |
Internet
```

How It Works — Step by Step

1. A packet leaves an instance destined for an external IP address.
2. The subnet's associated route table is checked for the **most specific matching** destination rule.
3. Local VPC traffic matches the automatic `local` route and stays inside the VPC.
4. Non-local traffic matches the `0.0.0.0/0` rule and is sent to whatever target is configured (IGW or NAT Gateway).
5. The target device (IGW/NAT Gateway) forwards the packet onward according to its own function.

Real-World Examples

- **Public web tier:** Route table sends all non-local traffic to the Internet Gateway so users can directly reach the web servers.
- **Private database tier:** Route table has no internet route at all (only the local route), fully isolating the database from any internet access.
- **Private application tier needing software updates:** Route table sends outbound traffic to a NAT Gateway so servers can download patches without being exposed inbound.

Use Cases

- Enforcing public vs private network behavior per subnet.
- Directing traffic to VPC peering connections or VPN/Direct Connect gateways.
- Creating fully isolated subnets with no internet route at all (for highly sensitive resources).

Advantages

- Fine-grained, explicit control over traffic direction per subnet.
- Central mechanism for enforcing the public/private network model.
- Supports advanced routing to peering connections, VPNs, and gateways.

Limitations / Disadvantages

- Misconfigured routes are a common source of connectivity issues (e.g. forgetting the NAT Gateway route).
- Route tables alone do not provide security filtering — they only control path, not permission (that is the job of Security Groups/NACLs).
- Each subnet can only be associated with one route table at a time.

Important Terminology

Term	Meaning
Route	A rule mapping a destination CIDR to a target
Local Route	Automatic route enabling intra-VPC communication
Main Route Table	Default table for the VPC, used by unassociated subnets
0.0.0.0/0	Catch-all destination representing "the entire internet"/anything not matched

Related AWS Services

- **Internet Gateway** — common target for public subnet routes.
- **NAT Gateway** — common target for private subnet outbound routes.
- **VPC Peering / Transit Gateway** — additional route targets for inter-VPC traffic.
- **Subnets** — the entities that route tables are associated with.

Important Relationships (Module 3 Context)

Route tables are the "decision layer" sitting between a subnet and the gateways: **Subnet → Route Table → Internet Gateway (public) / NAT Gateway (private)**. They directly determine the public/private classification discussed in the Subnets topic.

Commonly Confused: Main Route Table vs Custom Route Table

- Main route table is the automatic default; any subnet not explicitly associated elsewhere falls back to it.
- Custom route tables are created intentionally and must be explicitly associated with chosen subnets — this is the recommended production practice for predictable routing.

6. Security Groups

> **Definition:** A Security Group is a virtual, **stateful firewall** that controls inbound and outbound traffic at the instance (ENI) level for resources such as EC2 instances.

Purpose

- Provides fine-grained control over exactly what traffic is allowed to/from a specific resource.
- Acts as the last line of defense at the individual instance level, complementing subnet-level and route-level controls.

Core Concepts

- **Virtual firewall:** Security Groups exist logically at the AWS network layer, not as a physical device — but they behave functionally like a firewall attached to each resource.
- **Stateful firewall:** If inbound traffic is allowed in, the corresponding outbound response traffic is **automatically allowed** out, and vice versa — there is no need to write a matching return rule.
- **Inbound rules:** Define what traffic is allowed to reach the resource.
- **Outbound rules:** Define what traffic is allowed to leave the resource (default: all outbound traffic allowed).
- **Protocol:** The network protocol being filtered (TCP, UDP, ICMP, etc.).
- **Port:** The specific port number (or range) the rule applies to (e.g. 443 for HTTPS).
- **Source (inbound) / Destination (outbound):** Defines where allowed traffic may come from or go to — can be a specific IP/CIDR, or another Security Group.
- **IP-based rules:** Allow traffic from/to a specific IP address or CIDR range.
- **Security-group-based rules:** Allow traffic from/to any resource that belongs to a referenced Security Group (very useful for dynamic environments where IPs change).
- **Multiple security groups:** An instance can have multiple Security Groups attached; the **combined (union)** set of all their rules applies.
- **Default behavior:** A new Security Group denies all inbound traffic by default and allows all outbound traffic by default — you must explicitly add inbound rules to allow anything in.

Common Rule Examples

Rule Name	Protocol	Port	Meaning
HTTP	TCP	80	Allows unencrypted web traffic
HTTPS	TCP	443	Allows encrypted web traffic
SSH	TCP	22	Allows secure remote login (Linux)

- **Protocol (TCP):** Transmission Control Protocol — reliable, connection-oriented; used by HTTP, HTTPS, SSH.
- **Port:** A numbered "channel" that identifies which service on the instance should receive the traffic.
- **Why SSH should generally be restricted:**
  - Port 22 exposed to `0.0.0.0/0` invites brute-force login attempts from anywhere on the internet.
  - Best practice: restrict the SSH rule's source to a specific trusted IP range (e.g. office/VPN IP) or route access through a bastion host / Systems Manager Session Manager instead.

Stateful Behavior Explained

Stateful Traffic Flow Example

```
Client -----> (Inbound Rule: allow TCP 443) -----> EC2 Instance
Client <----- (Return traffic automatically allowed, no outbound rule needed) <----- EC2 Instance
```

Because Security Groups are stateful, once an inbound request is permitted, the response traffic tied to that same connection is automatically allowed back out — regardless of the outbound rules configured.

How It Works — Step by Step

1. Traffic arrives at the network interface of an EC2 instance.
2. AWS checks all Security Groups attached to that instance.
3. If **any** attached Security Group has a matching inbound "allow" rule, the traffic is permitted (Security Groups only support "allow" rules, never explicit "deny").
4. If no rule matches, the traffic is dropped by default.
5. Once allowed in, return traffic for that same connection is automatically permitted out due to statefulness.

Real-World Examples

- **Public web server:** Security Group allows inbound HTTP (80) and HTTPS (443) from `0.0.0.0/0`, and SSH (22) only from the company's office IP.
- **Private database server:** Security Group only allows inbound traffic on the database port (e.g. 3306 for MySQL) from the application tier's Security Group — not from any raw IP.
- **Internal microservice:** Security Group allows traffic only from other Security Groups belonging to trusted internal services, not from the public internet at all.

Use Cases

- Restricting SSH/RDP administrative access to trusted IPs only.
- Allowing an application tier's Security Group to reach a database tier's Security Group exclusively.
- Exposing only specific ports (80/443) to the public internet on web servers.

Advantages

- Simple, stateful, and easy to reason about (no need to write matching return rules).
- Can reference other Security Groups instead of hard-coded IPs, which scales well in dynamic environments.
- Applied at the instance/ENI level for precise, resource-specific control.

Limitations / Disadvantages

- Only supports "allow" rules — cannot explicitly deny/block specific traffic.
- Rule limits per Security Group and per network interface apply (soft limits, can be raised).
- Being stateful means it cannot be used to selectively block specific outbound response traffic once inbound is allowed.

Important Terminology

Term	Meaning
Stateful	Return traffic is automatically allowed once initiating traffic is permitted
Inbound Rule	Controls traffic coming into the resource
Outbound Rule	Controls traffic leaving the resource
ENI	Elastic Network Interface — the virtual network card a Security Group protects

Related AWS Services

- **EC2** — most common resource protected by Security Groups.
- **RDS / Lambda (in VPC)** — can also have Security Groups attached.
- **Network ACLs** — work alongside Security Groups at the subnet level.
- **Elastic Load Balancer** — typically has its own Security Group controlling access to backend instances.

Security Group vs Network ACL

Aspect	Security Group	Network ACL (NACL)
Level applied	Instance / ENI level	Subnet level
State	Stateful (return traffic auto-allowed)	Stateless (return traffic must be explicitly allowed)
Rule types	Allow rules only	Allow and Deny rules
Rule evaluation	All rules evaluated together (union)	Rules processed in numbered order, first match wins
Scope	Applies only to associated resources	Applies to all resources in the subnet

Important Relationships (Module 3 Context)

Security Groups sit at the innermost layer of the Module 3 traffic path: **VPC → Subnet → Route Table (gets traffic to the resource) → Security Group (decides if the resource accepts it)**. Even if routing allows traffic to physically reach an instance, the Security Group makes the final allow/deny decision.

> **Commonly Confused: Security Group vs Network ACL**
> - Security Group = stateful, instance-level, allow-only.
> - Network ACL = stateless, subnet-level, supports both allow and deny, evaluated in rule-number order.

7. NAT Gateway

> **Definition:** A NAT (Network Address Translation) Gateway is a managed AWS service that allows resources in a private subnet to initiate **outbound** connections to the internet (or other AWS services) while preventing the internet from initiating **inbound** connections to them.

Why NAT Is Needed

- Private subnet resources (e.g. app servers, databases) still often need outbound internet access — for software updates, calling external APIs, downloading patches.
- Without NAT, private resources would need a public IP and a direct IGW route to reach the internet, which would defeat the purpose of keeping them private/unreachable.
- NAT solves this by translating the private instance's private IP to the NAT Gateway's own address for outbound requests, while keeping the private instance itself unreachable from outside.

Core Concepts

- **Private subnet internet access:** Achieved indirectly — private instance → NAT Gateway → Internet Gateway → Internet.
- **Public subnet requirement:** A NAT Gateway itself must be deployed inside a **public subnet** (it needs a route to the IGW to reach the internet on behalf of private resources).
- **Elastic IP:** A NAT Gateway requires an Elastic IP address attached to it, which it uses as the source address for outbound traffic it forwards.
- **Route table relationship:** Private subnets must have a route sending `0.0.0.0/0` traffic to the NAT Gateway (not the IGW).
- **Outbound traffic:** Freely allowed — the private instance can initiate connections to the internet.
- **Inbound traffic:** Unsolicited connections from the internet are not allowed to reach the private instance through the NAT Gateway.

NAT Gateway Architecture

Private Instance Reaching the Internet via NAT

```
Private EC2
    |
Private Route Table  (0.0.0.0/0 -> NAT Gateway)
    |
NAT Gateway  (lives in a Public Subnet, has an Elastic IP)
    |
Public Route Table  (0.0.0.0/0 -> Internet Gateway)
    |
Internet Gateway
    |
Internet
```

How It Works — Step by Step

1. A private EC2 instance sends an outbound request (e.g. to download a software update).
2. The private subnet's route table sends the request to the NAT Gateway.
3. The NAT Gateway (sitting in a public subnet) translates the private IP source address to its own Elastic IP.
4. The NAT Gateway forwards the request through the Internet Gateway to the internet.
5. The response returns to the NAT Gateway's Elastic IP.
6. The NAT Gateway translates the destination back to the original private instance's IP and forwards the response.
7. The private instance receives the response — but no external party could have *initiated* a new connection to it.

Why Private Instances Can Initiate but Not Receive Connections

- **Outbound initiation works** because the NAT Gateway keeps track of the connection it started and knows where to route the response back.
- **Unsolicited inbound does not work** because there is no route table entry allowing the internet to target the private instance directly, and the NAT Gateway does not forward brand-new inbound connection attempts — only responses to connections the private side initiated.

Real-World Examples

- **Patch management:** Private application servers use a NAT Gateway to download OS/security patches without being exposed to inbound internet traffic.
- **Third-party API calls:** A private backend service calls an external payment gateway API through the NAT Gateway.
- **Package installation:** Private EC2 instances pull software packages (e.g. via a package manager) through NAT during deployment automation.

Use Cases

- Allowing private application/database servers limited, safe internet access.
- Supporting outbound-only integrations with external services.
- Maintaining strict inbound isolation for compliance-sensitive workloads while still enabling updates.

NAT Gateway vs Internet Gateway

Aspect	NAT Gateway	Internet Gateway
Direction	Outbound-only (from private resources)	Bidirectional (inbound + outbound)
Used by	Private subnets	Public subnets
Requires Elastic IP	Yes	No
Cost	Hourly + data processing charges	No additional charge
Makes resources publicly reachable?	No	Yes (if resource has a public IP)

NAT Gateway vs NAT Instance

Aspect	NAT Gateway	NAT Instance
Management	Fully managed by AWS	Self-managed EC2 instance running NAT software
Availability	Highly available within an AZ (managed, redundant internally)	Single point of failure unless manually configured for HA
Bandwidth	Scales automatically up to very high throughput	Limited by the chosen EC2 instance type
Maintenance	No patching/maintenance required	Requires OS patching and manual scaling
Cost model	Hourly + per-GB data processing	EC2 instance hourly cost
Security Groups	Not associated with a Security Group	Can be secured with Security Groups

Advantages

- Fully managed — no patching, scaling, or maintenance required.
- Highly available and automatically scales bandwidth.
- Keeps private resources shielded from inbound internet exposure.

Limitations / Disadvantages

- **Cost:** Charged hourly plus per-GB of data processed, which can add up for high-traffic workloads.
- **Availability:** A NAT Gateway is deployed in a single AZ; for full high availability, one NAT Gateway per AZ is recommended (with corresponding cost).
- Cannot be used to allow inbound connections from the internet — strictly outbound-initiated only.
- Adds a dependency: if the NAT Gateway or its AZ has issues, private subnet outbound access in that AZ is affected.

Important Terminology

Term	Meaning
NAT	Network Address Translation — translating private IPs to a shared public-facing IP
Elastic IP	Static public IP required by the NAT Gateway
Outbound-only access	Ability to initiate connections out, but not receive unsolicited ones in

Related AWS Services

- **Internet Gateway** — the NAT Gateway relies on the IGW to actually reach the internet.
- **Route Tables** — define which subnets send traffic to the NAT Gateway.
- **Subnets** — NAT Gateway sits in a public subnet, serves private subnets.
- **VPC** — the NAT Gateway is a component within a VPC's architecture.

Important Relationships (Module 3 Context)

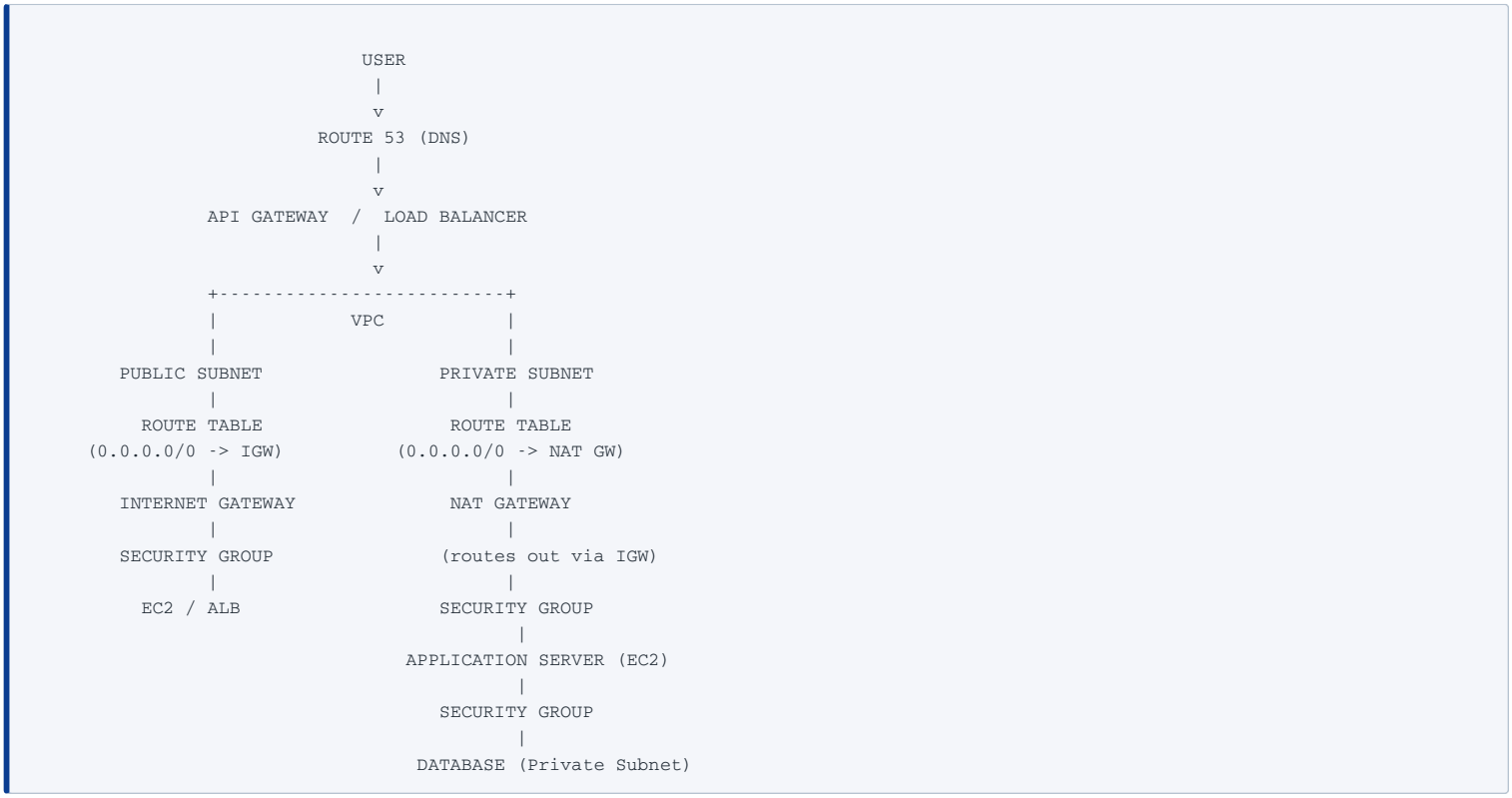
The NAT Gateway is the private-subnet counterpart to the Internet Gateway: **Private Subnet → Route Table → NAT Gateway (in Public Subnet) → Internet Gateway → Internet**. It exists specifically to give private resources safe, outbound-only internet access without compromising the isolation established by subnetting and routing.

Commonly Confused: Internet Gateway vs NAT Gateway

- Internet Gateway enables full bidirectional internet access for public subnets.
- NAT Gateway enables outbound-only internet access for private subnets and itself depends on an Internet Gateway to function.

8. How Module 3 Networking Fits Together

Complete Request Path Through Module 3 Services



Explanation of the Full Flow

- **Route 53** resolves the user's domain name to the entry point of the architecture (an API Gateway endpoint or a Load Balancer's Alias record).
- **API Gateway / Load Balancer** receives the request and forwards it into the VPC, applying auth, throttling, or routing logic as needed.
- **VPC** is the isolated network boundary containing all downstream infrastructure.
- **Public Subnet** hosts internet-facing resources (e.g. load balancer, bastion host); its **Route Table** sends internet traffic to the **Internet Gateway**.
- **Private Subnet** hosts application servers and databases; its **Route Table** sends outbound traffic to the **NAT Gateway**, which relies on the Internet Gateway to actually reach the internet.
- **Security Groups** are attached to individual resources (EC2 instances, databases) in both subnet tiers, providing the final, instance-level allow/deny decision regardless of which subnet the traffic is routed through.
- **Application Servers** in the private subnet process business logic and are only reachable from the load balancer/API Gateway layer, not directly from the internet.
- **Databases** sit in the most restricted private subnet tier, reachable only from the application server's Security Group.

Summary of Component Roles

Service	Role in the Architecture
Route 53	DNS resolution and traffic routing entry point
API Gateway	Managed, secured entry point for API-based backends
VPC	Isolated virtual network boundary for all resources
Subnets	Segment the VPC by AZ and by public/private tier
Route Tables	Decide where each subnet's traffic is directed
Internet Gateway	Enables bidirectional internet access for public subnets
NAT Gateway	Enables outbound-only internet access for private subnets
Security Groups	Instance-level stateful firewall — the final access decision

End of Module 3 – Part 1 study notes. These notes are concept-focused and intended for understanding architecture and relationships between AWS networking services, not for exam drills or console walkthroughs.